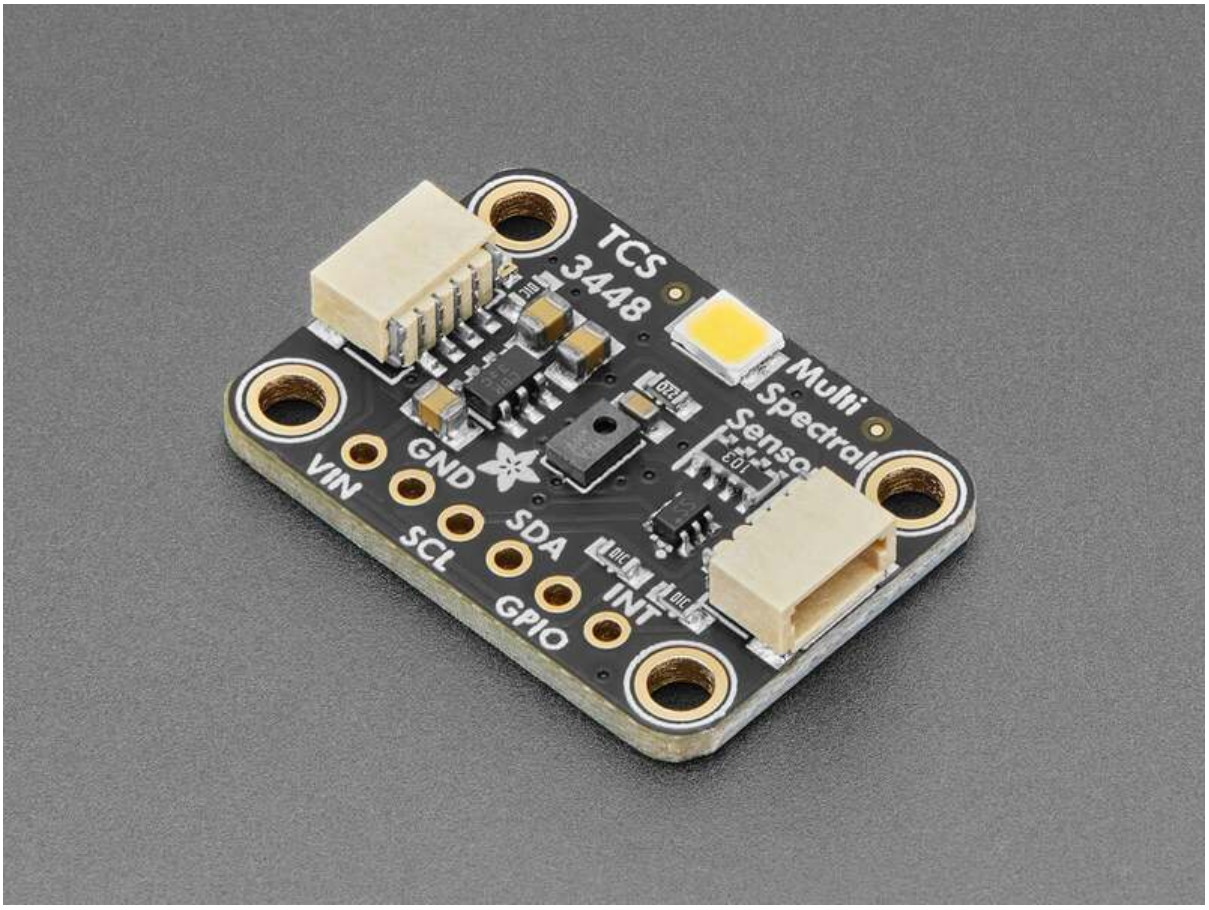
Adafruit TCS3448 14-Channel Light / Color Sensor Breakout

Created by Liz Clark



https://learn.adafruit.com/adafruit-tcs3448-14-channel-light-color-sensor-breakout

Last updated on 2026-09-29 02:28:24 PM UTC

Table of Contents

Overview 3

Pinouts 5

- Power Pins
- I2C Logic Pins
- Interrupt and GPIO Pin
- White LED
- Power LED and LED Jumper

CircuitPython and Python 7

- CircuitPython Microcontroller Wiring
- Python Computer Wiring
- Python Installation of TCS3448 Library
- CircuitPython Usage
- Python Usage
- Example Code

Python Docs 12

Arduino 12

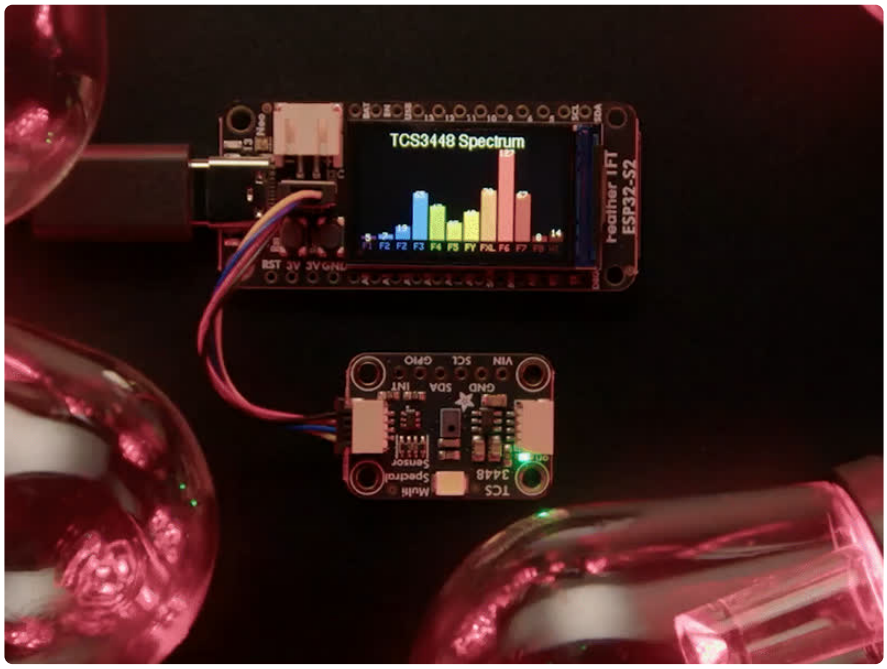
- Wiring
- Library Installation
- Example Code

Arduino Docs 16

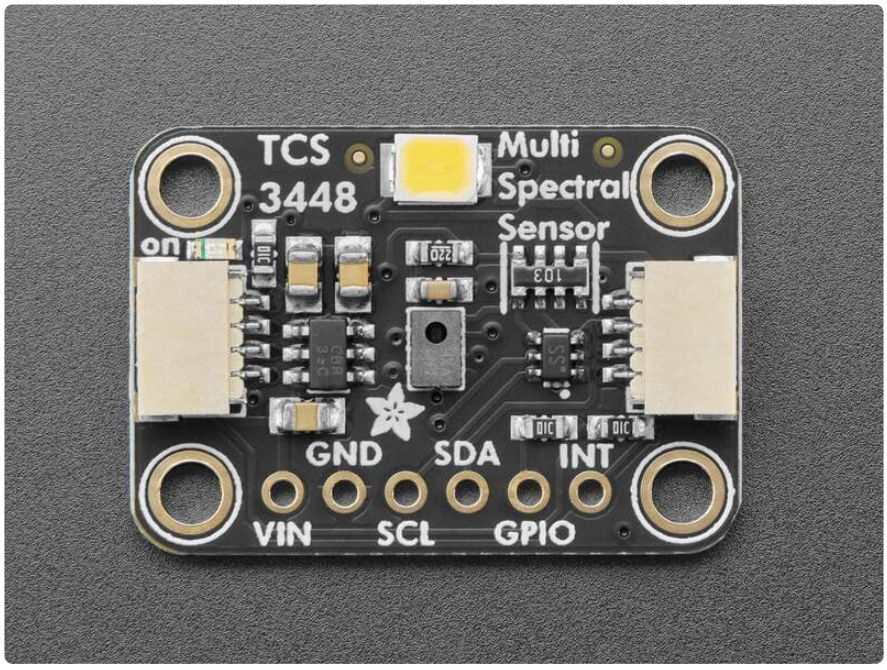
Downloads 16

- Files
- Schematic and Fab Print

Overview



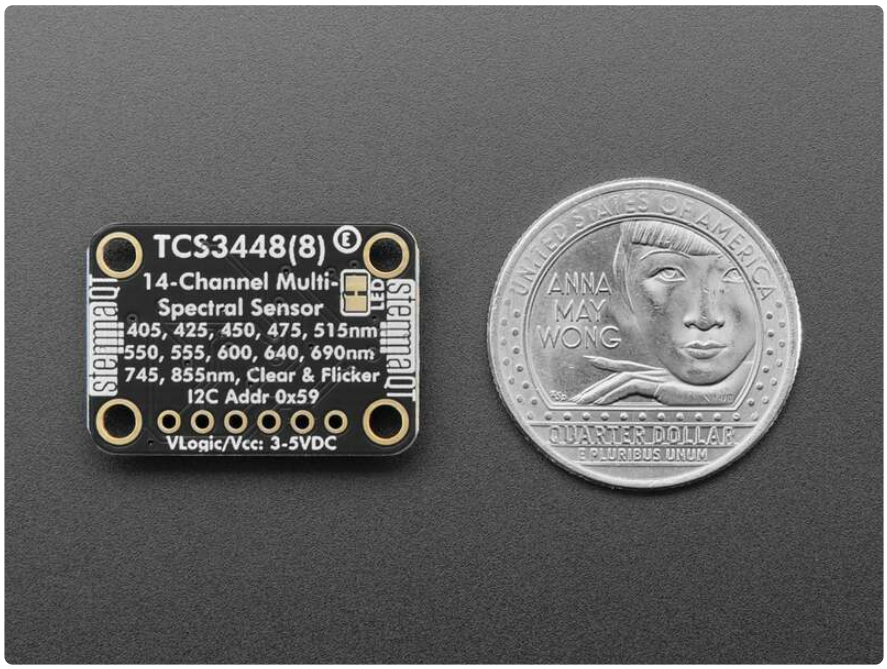
The **Adafruit TCS3448 (a.k.a. TCS34488) Multi-Spectral Sensor** is a multi-channel spectrometer: a light sensor that measures not only the total amount of light present but also how much light falls within specific wavelengths. This lets you detect which colors of light are present far more precisely than the human eye can.



The TCS3448 packs, within its 3x2mm footprint, 25 photodiodes that can detect 11 separate, overlapping bands of visible light, plus one near infrared, one clear, and a flicker channel for 14 channels total. Compared to the AS7341, this sensor has 4 more light channels - but you'll need to use a new firmware driver since it's not back-compatible.

The TCS3448 is ams Osram's 'refresh' of the AS7343, so it's an upgrade from the AS7341 and has some slight improvements/changes over the AS7343:

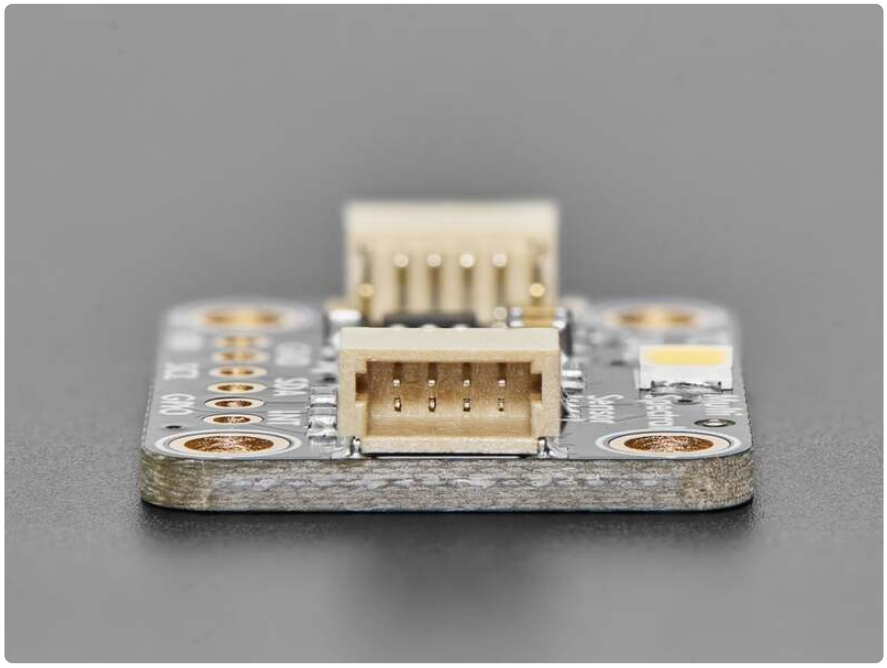
- I2C address changed from 0x39 to 0x59.
- Retuned optical filters: channel centers move by a few nanometers and several visible channels have higher responsivity especially violet/blue and the F8 far-red channel.
- Sensitivity is not higher everywhere; NIR response is lower.
- Firmware is not 100% drop-in! You'll need to recompile since the I2C address has changed



The multi-spectral color measuring capabilities of the TCS3448 can be used to quantify the specific makeup of light, either emitted or reflective. It's kind of like a mini spectroscopy sensor. This is possible thanks to the impressive collection of sensors in the TCS3448 being routed through 6 independent 16-bit ADCs that take the raw measurements and convert them to digital values that can be read out over I2C.

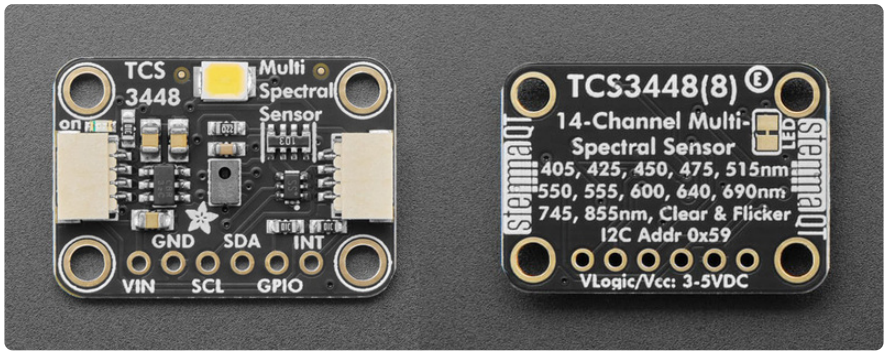
14 readable individual sensor elements (13 light channels plus flicker detection) don’t exactly fit through a 6-channel ADC all at once, so the chip includes a so-called Super

MUX (SMUX) that allows you to route the signal from any sensor to any ADC channel. The sensor also has GPIO and interrupt pins that can allow it to communicate directly with other sensors, or the microcontroller itself.



All of this capability is made accessible by mounting the sensor on a STEMMA QT form factor breakout board, complete with level shifting circuitry and SparkFun Qwiic () compatible Stemma QT () connectors. This means that you can, without needing to solder, connect our TCS3448 breakout into your 3.3V or 5V microcontroller of choice be it an Arduino Uno, Raspberry Pi, or one of the many CircuitPython-compatible boards (). While it certainly takes a bit of work to make all those different light sensors share their measurements, our Arduino and CircuitPython libraries take care of all of that hard work for you and even include example code to help get you started. **QT Cable is not included**, but we have a variety in the shop ().

Pinouts



The default I2C address is **0x59**.

Power Pins

- **VIN** - this is the power pin. To power the board, give it the same power as the logic level of your microcontroller - e.g. for a 5V micro like Arduino, use 5V.
- **GND** - common ground for power and logic

I2C Logic Pins

- **SCL** - I2C clock pin, connect to your microcontroller's I2C clock line. This pin can use 3-5V logic, and there's a **10K pullup** on this pin.
- **SDA** - I2C data pin, connect to your microcontroller's I2C data line. This pin can use 3-5V logic, and there's a **10K pullup** on this pin.
- STEMMA QT () **-** These connectors allow you to connectors to dev boards with **STEMMA QT** connectors or to other things with various associated accessories ().

Interrupt and GPIO Pin

- **INT** - the interrupt pin. It is an open-drain interrupt.
- **GPIO** - the GPIO pin on the breakout. It is an open-drain pin that can be used as an input or output.

White LED

On the front center of the board is a white LED. This LED is connected to an LED driver and it can be controlled using the CircuitPython or Arduino libraries. It can be used to shine white light onto an object and measure the colors in the light being reflected back from the object into the sensor.

Power LED and LED Jumper

- **Power LED -** In the upper left corner, above the STEMMA connector, on the front of the board, is the power LED, labeled **on**. It is a green LED.
- **LED jumper** - This jumper is located on the back of the board, in the right corner. Cut the trace on this jumper to cut power to the "on" LED.

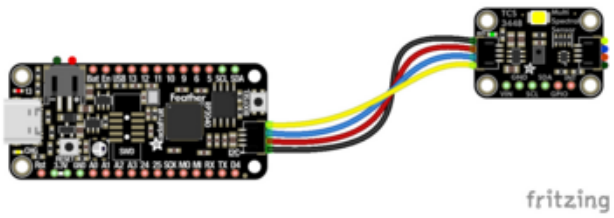
CircuitPython and Python

It's easy to use the **TCS3448** with Python or CircuitPython, and the Adafruit_CircuitPython_TCS3448 () module. This module allows you to easily write Python code to read data from the sensor.

You can use this driver with any CircuitPython microcontroller board or with a computer that has GPIO and Python thanks to Adafruit_Blinka, our CircuitPython-for-Python compatibility library ().

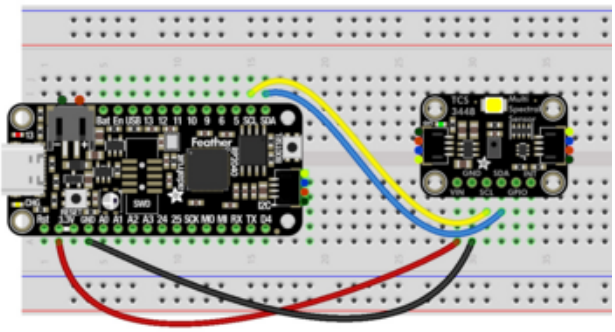
CircuitPython Microcontroller Wiring

First wire up a TCS3448 to your board exactly as shown below. Here's an example of wiring a Feather RP2040 to the sensor with I2C using one of the handy **STEMMA QT** ( ) connectors:



Board STEMMA 3V to **sensor VIN (red wire)**
Board STEMMA GND to **sensor GND (black wire)**
Board STEMMA SCL to **sensor SCL (yellow wire)**
Board STEMMA SDA to **sensor SDA (blue wire)**

The following is the TCS3448 wired to a Feather RP2040 using a solderless breadboard:

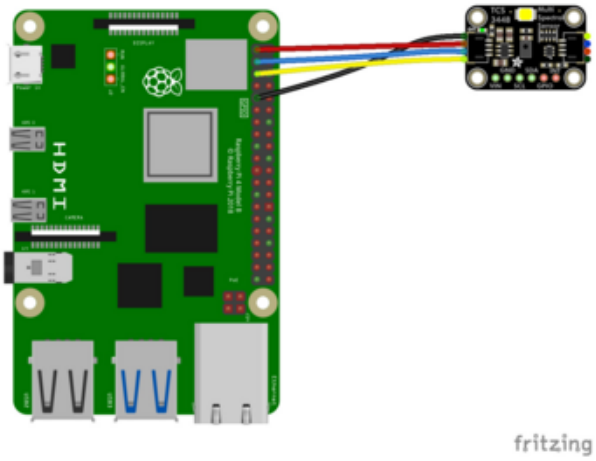


Board 3V to **sensor VIN (red wire)**
Board GND to **sensor GND (black wire)**
Board SCL to **sensor SCL (yellow wire)**
Board SDA to **sensor SDA (blue wire)**

Python Computer Wiring

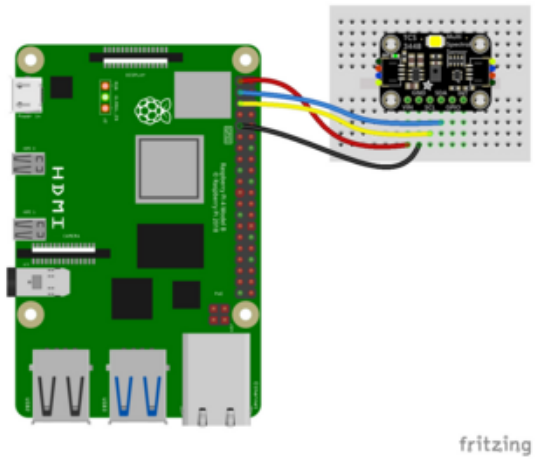
Since there are dozens of Linux computers/boards you can use, we will show wiring for Raspberry Pi. For other platforms, [please visit the guide for CircuitPython on Linux to see whether your platform is supported]() ().

Here's the Raspberry Pi wired with I2C using the STEMMA connector:



Pi 3V to **sensor VIN (red wire)**
Pi GND to **sensor GND (black wire)**
Pi SCL to **sensor SCL (yellow wire)**
Pi SDA to **sensor SDA (blue wire)**

Here's the Raspberry Pi wired with I2C using a solderless breadboard:



Pi 3V to **sensor VIN (red wire)**
Pi GND to **sensor GND (black wire)**
Pi SCL to **sensor SCL (yellow wire)**
Pi SDA to **sensor SDA (blue wire)**

Python Installation of TCS3448 Library

You'll need to install the **Adafruit_Blinka** library that provides the CircuitPython support in Python. This may also require enabling I2C on your platform and verifying you are running Python 3. Since each platform is a little different, and Linux changes often, please visit the CircuitPython on Linux guide to get your computer ready ()!

Once that's done, from your command line run the following command:

- `pip3 install adafruit-circuitpython-tcs3448`

If your default Python is version 3 you may need to run 'pip' instead. Just make sure you aren't trying to use CircuitPython on Python 2.x, it isn't supported!

CircuitPython Usage

To use with CircuitPython, you need to first install the **Adafruit_CircuitPython_TCS3448** library, and its dependencies, into the **lib** folder on your **CIRCUITPY** drive. Then you need to update **code.py** with the example script.

Thankfully, we can do this in one go. In the example below, click the **Download Project Bundle** button below to download the necessary libraries and the **code.py** file in a zip file. Extract the contents of the zip file, and copy the **entire lib folder** and the **code.py** file to your **CIRCUITPY** drive.

Your **CIRCUITPY/lib** folder should contain the following folders and file:

- **adafruit_bus_device/**
- **adafruit_register/**
- **adafruit_tcs3448.mpy**



Python Usage

Once you have the library `pip3` installed on your computer, copy or download the following example to your computer, and run the following, replacing **code.py** with whatever you named the file:

```
python3 code.py
```

Example Code

If running CircuitPython: Once everything is saved to the **CIRCUITPY** drive, connect to the serial console () to see the data printed out!

If running Python: The console output will appear wherever you are running Python.

```
# SPDX-FileCopyrightText: Copyright (c) 2026 Liz Clark for Adafruit Industries
#
# SPDX-License-Identifier: MIT
import time

import board

from adafruit_tcs3448 import TCS3448

i2c = board.I2C()
sensor = TCS3448(i2c)

CHANNEL_LABELS = [
    "FZ (450nm blue)",
    "FY (560nm yellow-green)",
    "FXL (596nm orange)",
    "NIR (855nm near-IR)",
    "VIS_TL_0 (clear top-left, cycle 1)",
    "VIS_BR_0 (clear btm-right, cycle 1)",
    "F2 (424nm violet-blue)",
    "F3 (473nm blue-cyan)",
    "F4 (516nm green)",
    "F6 (636nm red)",
```

```
    "VIS_TL_1 (clear top-left, cycle 2)",
    "VIS_BR_1 (clear btm-right, cycle 2)",
    "F1 (407nm violet)",
    "F7 (687nm deep red)",
    "F8 (748nm near-IR edge)",
    "F5 (546nm green-yellow)",
    "VIS_TL_2 (clear top-left, cycle 3)",
    "VIS_BR_2 (clear btm-right, cycle 3)",
]

while True:
    readings = sensor.all_channels
    print("--- TCS3448 Channel Readings ---")
    for label, value in zip(CHANNEL_LABELS, readings):
        print(f"  {label}: {value}")
    print()
    time.sleep(1)
```

https://github.com/adafruit/Adafruit_CircuitPython_TCS3448/blob/main/examples/tcs3448_simpletest.py

First, the sensor gets initialized over I2C. In the main loop the spectral data is read from the sensor and the different channel values are printed to the serial console once per second.

```
Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.
code.py output:
--- TCS3448 Channel Readings ---
  FZ (450nm blue): 171
  FY (560nm yellow-green): 480
  FXL (596nm orange): 505
  NIR (855nm near-IR): 202
  VIS_TL_0 (clear top-left, cycle 1): 666
  VIS_BR_0 (clear btm-right, cycle 1): 17942
  F2 (424nm violet-blue): 87
  F3 (473nm blue-cyan): 184
  F4 (516nm green): 286
  F6 (636nm red): 559
  VIS_TL_1 (clear top-left, cycle 2): 668
  VIS_BR_1 (clear btm-right, cycle 2): 17932
  F1 (407nm violet): 42
  F7 (687nm deep red): 410
  F8 (748nm near-IR edge): 112
  F5 (546nm green-yellow): 119
  VIS_TL_2 (clear top-left, cycle 3): 664
  VIS_BR_2 (clear btm-right, cycle 3): 17930

--- TCS3448 Channel Readings ---
  FZ (450nm blue): 181
  FY (560nm yellow-green): 135
  FXL (596nm orange): 100
  NIR (855nm near-IR): 609
  VIS_TL_0 (clear top-left, cycle 1): 394
  VIS_BR_0 (clear btm-right, cycle 1): 10892
  F2 (424nm violet-blue): 102
  F3 (473nm blue-cyan): 171
  F4 (516nm green): 144
  F6 (636nm red): 119
  VIS_TL_1 (clear top-left, cycle 2): 382
  VIS_BR_1 (clear btm-right, cycle 2): 10472
  F1 (407nm violet): 69
  F7 (687nm deep red): 231
  F8 (748nm near-IR edge): 223
  F5 (546nm green-yellow): 35
  VIS_TL_2 (clear top-left, cycle 3): 373
  VIS_BR_2 (clear btm-right, cycle 3): 10153
```

Python Docs

Python Docs ()

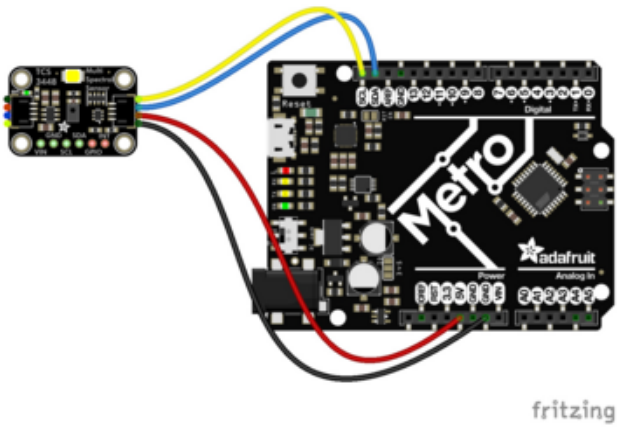
Arduino

Using the TCS3448 breakout with Arduino involves wiring up the breakout to your Arduino-compatible microcontroller, installing the Adafruit_TCS3448 () library, and running the provided example code.

Wiring

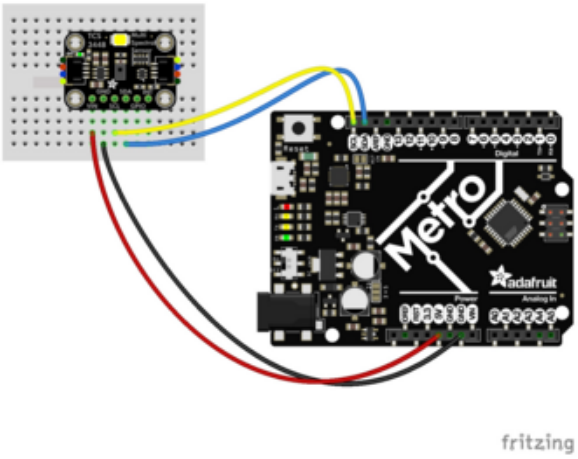
Wire as shown for a **5V** board like an Uno. If you are using a **3V** board, like an Adafruit Feather, wire the board's 3V pin to the sensor **VIN**.

Here is an Adafruit Metro wired up to the sensor using the STEMMA QT connector:



Board 5V to **breakout VIN (red wire)**
Board GND to **breakout GND (black wire)**
Board SCL to **breakout SCL (yellow wire)**
Board SDA to **breakout SDA (blue wire)**

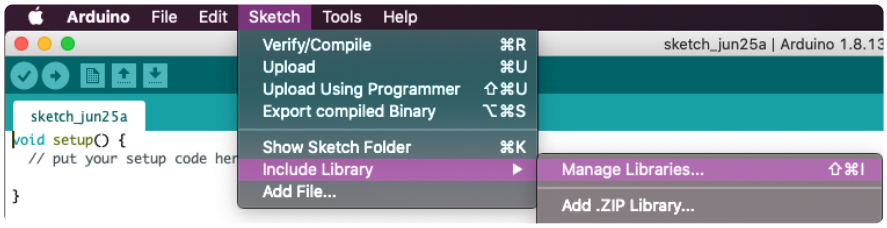
Here is an Adafruit Metro wired up using a solderless breadboard:



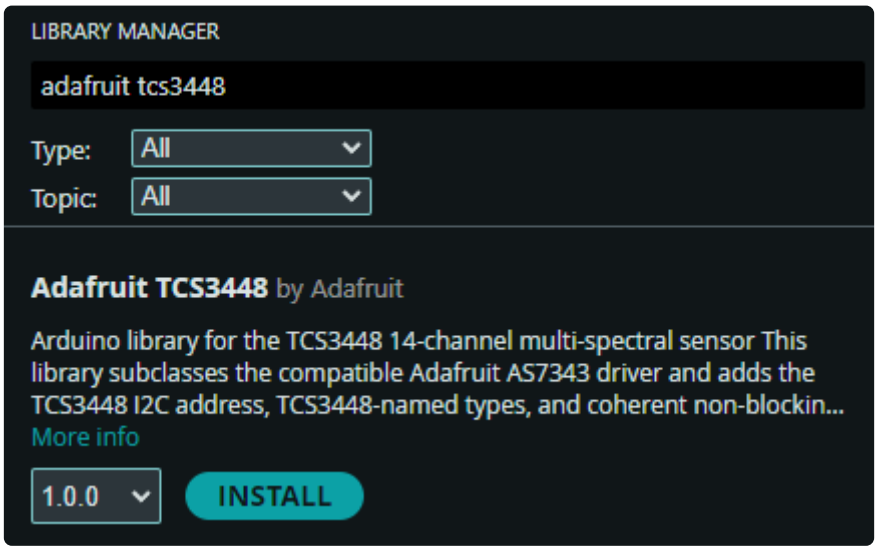
Board 5V to **breakout VIN (red wire)**
Board GND to **breakout GND (black wire)**
Board SCL to **breakout SCL (yellow wire)**
Board SDA to **breakout SDA (blue wire)**

Library Installation

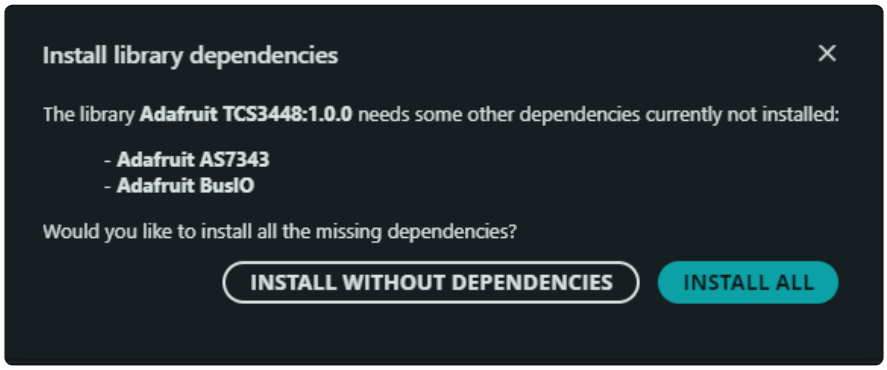
You can install the **Adafruit_TCS3448** library for Arduino using the Library Manager in the Arduino IDE.



Click the **Manage Libraries ...** menu item, search for **Adafruit TCS3448**, and select the **Adafruit TCS3448** library:



If asked about dependencies, click "Install all".



If the "Dependencies" window does not come up, then you already have the dependencies installed.

Text emphasized with a yellow exclamation:

If the dependencies are already installed, you must make sure you update them through the Arduino Library Manager before loading the example!

Example Code

```
/*!
 * @file basic_readings.ino
 *
 * Basic example for the Adafruit TCS3448 14-Channel Multi-Spectral Sensor.
 *
 * Reads all 18 results from the three automatic SMUX cycles and prints the
 * 12 spectral channels plus one broadband visible channel.
 */

#include <Adafruit_TCS3448.h>

Adafruit_TCS3448 tcs3448;

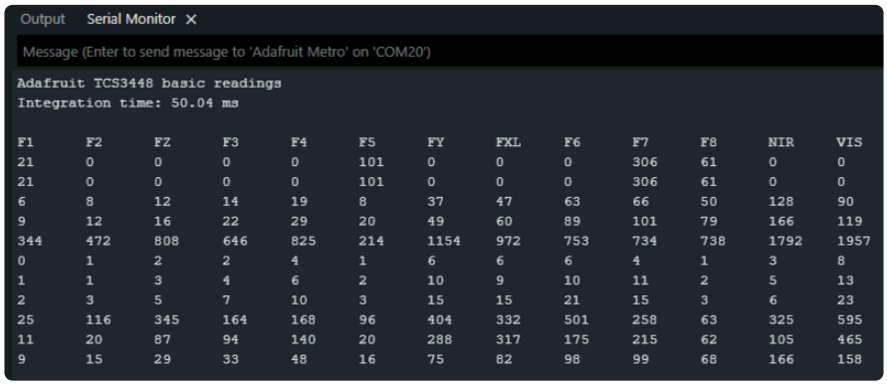
void setup() {
  Serial.begin(115200);
  // Wait for the Serial Monitor on native USB boards. Remove this loop to let
  // the sketch run immediately from battery power.
  while (!Serial) {
    delay(10);
  }
  delay(250);
```

```
22
23    Serial.println(F("Adafruit TCS3448 basic readings"));
24
25    if (!tcs3448.begin()) {
26      Serial.println(F("Could not find a TCS3448 sensor!"));
27      while (true) {
28        delay(10);
29      }
30    }
31
32    if (!tcs3448.setGain(TCS3448_GAIN_64X) || !tcs3448.setATIME(29) ||
33        !tcs3448.setASTEP(599) ||
34        !tcs3448.setSMUXMode(TCS3448_SMUX_18CH)) {
35      Serial.println(F("Could not configure the sensor!"));
36      while (true) {
37        delay(10);
38      }
39    }
40
41    Serial.print(F("Integration time: "));
42    Serial.print(tcs3448.getIntegrationTime());
43    Serial.println(F(" ms"));
44    Serial.println();
45    Serial.println(F("F1\tF2\tFZ\tF3\tF4\tF5\tFY\tFXL\tF6\tF7\tF8\tNIR\tVIS"));
46  }
47
48  void loop() {
49    uint16_t readings[TCS3448_CHANNEL_COUNT];
50
51    if (tcs3448.readAllChannels(readings)) {
52      Serial.print(readings[TCS3448_CHANNEL_F1]);
53      Serial.print('\t');
54      Serial.print(readings[TCS3448_CHANNEL_F2]);
55      Serial.print('\t');
56      Serial.print(readings[TCS3448_CHANNEL_FZ]);
57      Serial.print('\t');
58      Serial.print(readings[TCS3448_CHANNEL_F3]);
59      Serial.print('\t');
60      Serial.print(readings[TCS3448_CHANNEL_F4]);
61      Serial.print('\t');
62      Serial.print(readings[TCS3448_CHANNEL_F5]);
63      Serial.print('\t');
64      Serial.print(readings[TCS3448_CHANNEL_FY]);
65      Serial.print('\t');
66      Serial.print(readings[TCS3448_CHANNEL_FXL]);
67      Serial.print('\t');
68      Serial.print(readings[TCS3448_CHANNEL_F6]);
69      Serial.print('\t');
70      Serial.print(readings[TCS3448_CHANNEL_F7]);
71      Serial.print('\t');
72      Serial.print(readings[TCS3448_CHANNEL_F8]);
73      Serial.print('\t');
74      Serial.print(readings[TCS3448_CHANNEL_NIR]);
75      Serial.print('\t');
76      Serial.println(readings[TCS3448_CHANNEL_VIS_TL_0]);
77    }
```

```
78
79    delay(500);
80  }
```

https://github.com/adafruit/Adafruit_TCS3448/blob/main/examples/basic_readings/basic_readings.ino

Upload the sketch to your board and open up the Serial Monitor (**Tools -> Serial Monitor**) at 115200 baud. You'll see the TCS3448 recognized over I2C. Then, the spectral reading values of different channels will be printed out to the Serial Monitor.



Arduino Docs

Arduino Docs ()

Downloads

Files

- TCS3448 Datasheet (http://adafru.it/652565253448001121200)
- EagleCAD PCB files on GitHub ()
- Fritzing object in the Adafruit Fritzing Library ()

Schematic and Fab Print

